Theming and a modern skin

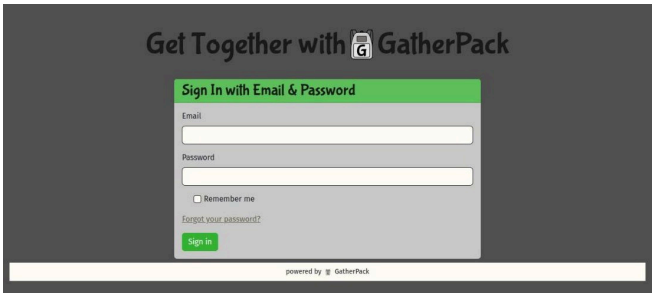
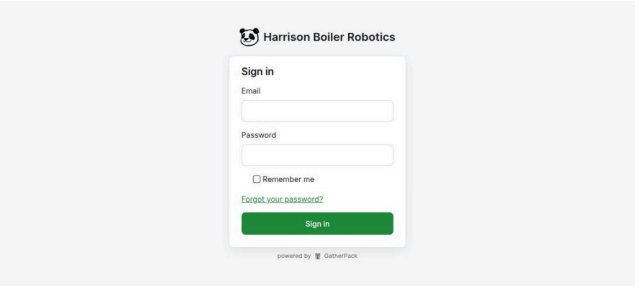
Summary

Replace the free-text theme page with a few real choices (brand color, logo, font, corner style), and have GatherPack work out every other color from them. Add a modern default look and a redesigned sign-in page. All of it sits on top of the existing pages: controllers, models, permissions, hooks and page markup stay as they are, and one setting switches back to the old look.

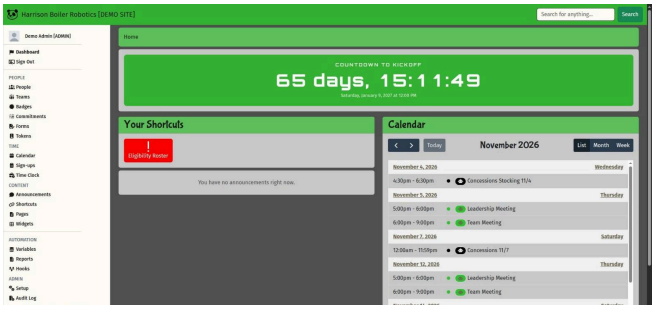
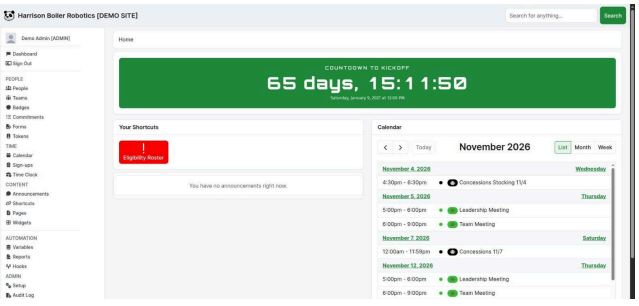
The look

These are mockups, not a final design. The new look is CSS applied in a browser to a demo copy of the site. The sign-in logo line and heading were also edited by hand, to show what a replacement template would do.

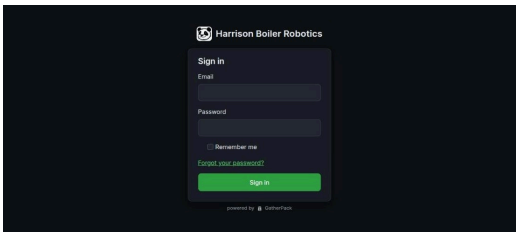
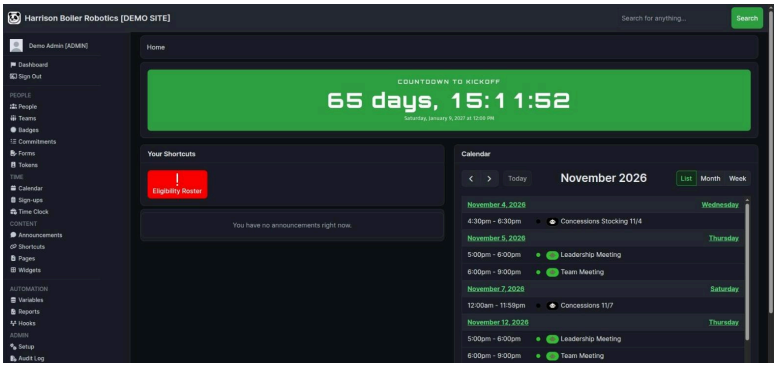
Sign-in page

Today	New look
	

Dashboard

Today	New look
	

Dark mode

Sign-in page	Dashboard
	

Problem

The sign-in page is the first thing anyone sees, and it looks dated: a large novelty-font slogan over a gray card. It also ignores the theme. It always shows the GatherPack logo and name, even when the site has its own logo and title.

The theme page looks like it controls color, but it doesn't reach most of them. An admin types CSS variables into a text box, with no preview and no check that the text stays readable. Setting `--bs-primary` to green leaves these unchanged:

- **Links and `.bg-primary`.** They read `--bs-primary-rgb` and `--bs-link-color-rgb`, which stay at the original greige.
- **Outline buttons and pagination.** Their colors were fixed when the stylesheet was built. The original primary appears as a literal value 32 times in the compiled CSS, plus 13 times as an RGB triplet.
- **Search and other success buttons.** They stay Bootstrap's green, so a different green sits right beside the admin's chosen one.
- **Calendar buttons.** They stay FullCalendar's navy.
- **Comboboxes.** They keep their own fixed colors.

The result is several unrelated color families on one screen, however carefully the admin chooses. The variable reference on the theme page has also drifted: it lists a page background default that the stylesheet no longer uses. Dark mode styles are compiled, but the theme switcher partial is never rendered, so nobody can reach them.

What it does

- **Choose, don't type.** The theme page offers brand color, logo, font and corner style, with a live preview of real components beside them.
- **Readable by default.** Hover, focus, tint and text-on-color values are derived from the brand color. If a choice can't reach readable contrast, a warning appears just above Save.
- **One color everywhere.** The brand color reaches buttons, links, focus rings, pagination, the calendar and comboboxes.
- **A modern default look.** Neutral page background, white cards with a thin border, a current system font, consistent spacing and quieter navigation.

- **A real sign-in page.** A focused single-column page using the site's own logo and title, with the same fields and sign-in providers.
- **Try before switching.** An admin can preview the new look in their own session. Everyone else sees the old look until the setting is on.
- **Advanced options stay.** The CSS variables and custom CSS fields still work and apply last, so existing themes keep working.
- **Dark mode.** Each person chooses Light, Dark or Auto, and Auto follows their device. The site's brand color carries over to both palettes. Printed pages stay light.

How it's built

The views are already separate from the logic, and they describe elements by meaning (`card`, `btn-primary`, `table`). There are 420 templates and about 1,200 card classes, but only 4 view files with inline colors. That makes the Bootstrap class names a good contract to keep. The change goes underneath them, in three layers:

1. **Generated palette.** New `Theme` columns hold the choices. Ruby derives the full set of Bootstrap, FullCalendar and combobox variables from them, including the RGB triplets that CSS can't compute at runtime from a hex variable. `layouts/_theme_styles`, which the application, sign-in and kiosk layouts already render, outputs the result ahead of any admin overrides.
2. **Skin stylesheet.** A separate compiled stylesheet restyles the existing classes and is linked only when the setting is on. With it off, the page gets the same CSS as today, so turning it off is a full rollback.
3. **Template overrides.** A skin folder is added to the view path for each request when the setting is on (`prepend_view_path`). A file there replaces the template of the same name, so the sign-in page, navigation and dashboard can change their markup without editing the originals. Everything else falls through unchanged. A fork check lists any overridden template whose original has changed upstream.

A browser-only trial injected about 5 KB of CSS into a demo copy of the site, with no markup changes, and restyled the dashboard and the sign-in page convincingly. The sign-in page's fixed slogan, logo and heading still need a replacement template. Forms, tables, the ledger, the kiosk and phone widths weren't part of the trial.

Not recommended:

- **A separate front end (React or similar).** It would mean rewriting 420 templates and re-creating permission checks that views get for free today, for a result users could not tell apart from this one.
- **Replacing Bootstrap with Tailwind.** Every template would change, and so would the 54 test assertions that select on these class names, making every upstream merge a conflict.

Size: Medium, mostly new files. One reversible migration, 4 upstream files (the CSS build script, the theme partial, the theme edit page, the theme controller's permitted fields), flag off by default. Each screen redesigned after that is one new override file.

Risks

- **Content that carries its own colors.** Team and event type colors are chosen by admins and may clash or be hard to read. Pages and widgets are stored as HTML with their own inline styles, and need

a review pass.

- **Coverage.** Forms, the ledger, the kiosk, tables and modals have their own styles. Each needs checking at phone width, and printed reports must stay as they are.
- **Override drift.** An overridden template can miss a later upstream change. Keep overrides few, and rely on the fork check.
- **Looks are not flow.** A skin makes pages look current but doesn't make a confusing page clear. Screens that need that get their own override, one at a time.

First step

Build the skin stylesheet and the sign-in override behind the flag, using hard-coded light and dark palettes. The skin uses color variables from the start, so both palettes share one set of rules. Review it across the main screens on a staging copy, in both modes, at desktop and phone widths, before building the generator and the new theme page. This tests the look first, which is the part most likely to change, and it ships the most visible improvement early.

Adoption

The generated palette and new theme page fix a problem in core and could be adopted without the skin. The skin and overrides are independent of the other features in this folder.